

11. Implement interface for area calculation for different shapes.

```
interface Shape {
    double area();
}

class Circle implements Shape {
    double radius;
    Circle(double r) { radius = r; }
    public double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    double l, b;
    Rectangle(double l, double b) { this.l = l; this.b = b; }
    public double area() { return l * b; }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Shape c = new Circle(5);
        Shape r = new Rectangle(4, 6);
        System.out.println("Circle Area = " + c.area());
        System.out.println("Rectangle Area = " + r.area());
    }
}
```

12. Create a package called Arithmetic.

```
// File: Arithmetic/Operations.java
package Arithmetic;

public class Operations {
    public int add(int a, int b){ return a+b; }
    public int sub(int a, int b){ return a-b; }
    public int mul(int a, int b){ return a*b; }
    public double div(int a, int b){ return (double)a/b; }
    public int mod(int a, int b){ return a%b; }
}

// File: UsePackage.java
import Arithmetic.Operations;

public class UsePackage {
    public static void main(String[] args){
        Operations op = new Operations();
        System.out.println("Add = " + op.add(10,5));
        System.out.println("Sub = " + op.sub(10,5));
        System.out.println("Mul = " + op.mul(10,5));
        System.out.println("Div = " + op.div(10,5));
        System.out.println("Mod = " + op.mod(10,5));
    }
}
```

13. Two threads: Fibonacci & Perfect numbers.

```
class Fibonacci extends Thread {
    public void run() {
        int n1=0,n2=1,n3;
        System.out.println("Fibonacci series:");
        System.out.print(n1+" "+n2+" ");
        for(int i=2;i<10;i++){
            n3=n1+n2;
            System.out.print(n3+" ");
            n1=n2; n2=n3;
        }
        System.out.println();
    }
}

class PerfectNumbers extends Thread {
    int start, end;
    PerfectNumbers(int s,int e){ start=s; end=e; }
    public void run(){
        System.out.println("Perfect numbers:");
    }
}
```

```

        for(int n=start;n<=end;n++){
            int sum=0;
            for(int i=1;i<=n/2;i++) if(n%i==0) sum+=i;
            if(sum==n) System.out.print(n+" ");
        }
        System.out.println();
    }
}

public class ThreadDemo {
    public static void main(String[] args){
        new Fibonacci().start();
        new PerfectNumbers(1,500).start();
    }
}

```

14. Custom Exception for Marks Out of Bound.

```

class MarksOutOfBounds extends Exception {
    MarksOutOfBounds(String msg){ super(msg); }
}

public class ExceptionDemo {
    public static void main(String[] args){
        int marks = 120;
        try {
            if(marks > 100)
                throw new MarksOutOfBounds("Marks Out of Bound!");
            else
                System.out.println("Marks = " + marks);
        } catch(MarksOutOfBounds e){
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

```

15. Wrapper Class Demo.

```

public class WrapperDemo {
    public static void main(String[] args){
        Integer i = Integer.valueOf(100);    // Boxing
        int n = i.intValue();                 // Unboxing

        System.out.println("Integer = " + i);
        System.out.println("Binary of 10 = " + Integer.toBinaryString(10));
        System.out.println("Max int = " + Integer.MAX_VALUE);
        System.out.println("Min int = " + Integer.MIN_VALUE);
        System.out.println("Parse Int from String = " + Integer.parseInt("123"));
    }
}

```

16. File Processing using Byte Stream.

```

import java.io.*;

public class ByteFileDemo {
    public static void main(String[] args) throws Exception {
        FileOutputStream fout = new FileOutputStream("bytefile.txt");
        String data = "Hello using Byte Stream";
        fout.write(data.getBytes());
        fout.close();

        FileInputStream fin = new FileInputStream("bytefile.txt");
        int ch;
        while((ch=fin.read())!=-1) System.out.print((char)ch);
        fin.close();
    }
}

```

17. File Processing using Character Stream.

```

import java.io.*;

```

```

public class CharFileDemo {
    public static void main(String[] args) throws Exception {
        FileWriter fw = new FileWriter("charfile.txt");
        fw.write("Hello using Character Stream");
        fw.close();

        FileReader fr = new FileReader("charfile.txt");
        int ch;
        while((ch=fr.read())!=-1) System.out.print((char)ch);
        fr.close();
    }
}

```

18. Applet to draw Shapes.

```

import java.applet.Applet;
import java.awt.*;

/* <applet code="ShapeApplet" width=400 height=400></applet> */

public class ShapeApplet extends Applet {
    public void paint(Graphics g){
        g.drawLine(50,200,100,100);
        g.drawLine(150,200,100,100);
        g.drawOval(50,200,100,30);

        g.drawOval(200,100,80,30);
        g.drawLine(200,115,200,200);
        g.drawLine(280,115,280,200);
        g.drawOval(200,185,80,30);

        g.drawOval(50,250,100,100);
        g.drawRect(70,270,60,60);

        g.drawRect(200,250,100,100);
        g.drawOval(200,250,100,100);
    }
}

```

19. Applet - Calculator.

```

import java.applet.*;
import java.awt.*;
import java.awt.event.*;

/* <applet code="CalcApplet" width=300 height=300></applet> */

public class CalcApplet extends Applet implements ActionListener {
    TextField t1, t2, result;
    Button add, sub, mul, div;

    public void init(){
        t1 = new TextField(5);
        t2 = new TextField(5);
        result = new TextField(10);
        add = new Button("+"); sub = new Button("-");
        mul = new Button("*"); div = new Button("/");

        add(t1); add(t2); add(result);
        add(add); add(sub); add(mul); add(div);

        add.addActionListener(this);
        sub.addActionListener(this);
        mul.addActionListener(this);
        div.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e){
        int a = Integer.parseInt(t1.getText());
        int b = Integer.parseInt(t2.getText());
        int res=0;
        if(e.getSource()==add) res=a+b;
        if(e.getSource()==sub) res=a-b;
        if(e.getSource()==mul) res=a*b;
        if(e.getSource()==div) res=a/b;
    }
}

```

```

        result.setText(""+res);
    }
}

```

20. Applet - Ball Animation.

```

import java.applet.*;
import java.awt.*;

/* <applet code="BallApplet" width=400 height=200></applet> */

public class BallApplet extends Applet implements Runnable {
    int x=0, y=100, dx=5;
    Thread t;

    public void init(){
        t = new Thread(this);
        t.start();
    }

    public void run(){
        while(true){
            x += dx;
            if(x>getWidth()-50 || x<0) dx = -dx;
            repaint();
            try{ Thread.sleep(50); } catch(Exception e){}
        }
    }

    public void paint(Graphics g){
        g.setColor(Color.red);
        g.fillOval(x,y,50,50);
    }
}

```